

GPT4o-mini-based Mutant Selection for Java Mutation Testing

Pham Manh Quynh, Tran Truc Vy, Le Nguyen Dinh Khoi, Nguyen Truong Vinh, Nguyen Nhat Anh
FPT University, HCM, VietNam

Abstract—

Index Terms—component, formatting, style, styling, insert

GPT-4o-mini can select a mutant subset better than random under the same budget.

I. INTRODUCTION

Mutation testing is a technique to evaluate a test suit by creating mutants and test the ability whether the test suit can recognize them. But mutation testing often creates an enormous numbers of mutant. Ojdanic et al. (2023) report for over 10 million mutants for 5 Apache Commons Projects and Jimenez et al. (2018) works with more than 100,000 mutants for 5 Defect4J projects. The huge number of mutant significantly increases the practical cost and make mutation testing hard to apply in practice.

Recent studies have shown the need to reduce the number of mutants through mutant selection to balance the cost and testing effectiveness. Two branches of solutions have emerged. One is to use machine learning to pick and predict mutants: Cerebro (TSE 2023) reduced 68% equivalent mutants and 90% of testing time by applying machine learning to predict subsuming mutants and select the important mutants. Another group uses LLMs: Wang et al. (2025) showed that LLMs achieved a high bug detection rate 76.47% compared to 44.15% of rule-based technique; SMART (Wang et al., 2026) increased validity from 42.89% to 65.6% ; PRIMG (2025) used LLMs for test generation and mutant prioritization in mutation testing. At the same time. Random selection is still used widely as a baseline because of the simplicity and reproducibility.

However, no study has evaluated GPT-4o-mini as a mutant selection decision maker against a random baseline under an identical mutant budget. The common thread in the above LLMs studies is that they use LLM to create mutants, filter equivalent mutants and generate tests. There is no research that uses LLM as a mutant selector a subset and compares it with a random selection under the same mutant budget and the same metrics. The closest research that matches the design is Jimenez et al. (2018) but chose mutant base on naturalness using n-gram model and compare it using the Wilcoxon signed-rank test with effect size, but concluded that naturalness-based selection is only equivalent to (slightly inferior to) random selection, with small effect size, and they used n-gram, not LLM. On the other hand, the trained models all admitted to a drop in performance when transitioning to a new project. The remaining gap is if a training-free LLM like

In this paper experimentally evaluate the GPT-4o-mini as a mutant selector, compare it with random fixed-seed selection with the same mutant budget which is top 10% of the mutant pool. We ran PIT on two Defects4J projects Commons Lang and Commons Math, we have collected 50,694 mutant with determined ground-truth killed/survived. GPT-4o-mini graded the usefulness of each mutant; both GPT and random chose the same top 10% in each group (project, version, class), then compared the mutation score in pairs by Wilcoxon signed-rank one-sided and matched-pairs rank-biserial correlation. All metrics, statistical test, thresholds and subgroup rules defined before excuting the experiment to avoid HARKing. The result showed that GPT-4o-mini outperformed random selection in Commons Lang, but there was not enough evidence in Commons Math and we quantitatively analyzed the cause of this negative result instead of ignoring it.

To summarize, this paper contributes:

- 1) The first empirical evaluation of GPT-4o-mini-based mutant selection for selecting a reduced mutant subset under a fixed budget using mutation score (paired per project/version/class), Wilcoxon signed rank and match-pairs rank-biserial correlation on Defects4J
- 2) Experimental results showing that GPT-4o-mini significantly outperforms random fixed-seed selection on Commons Lang, but not on Commons Math, where a post-hoc power analysis shows the design retains only approximately 14% power even under a Lang-sized effect.
- 3) A publicly available replication package - dataset, prompts, pipeline and analysis notebooks at <https://github.com/KhoiPhanDinh/RT-SWT-003-nhom3>

The rest of this paper is structured as follows: Section 2 examines related work in two branches: methods for selecting mutants based on trained machine learning models, and LLM applications in mutation testing, and points out the gap that this research fills. Section 3 describes the methodology including Defects4J dataset and mutant generation using PIT, mutant selection pipeline for GPT-4o-mini and random fixed-seed selection under the same mutant budget, the mutation score metric, statistical tests and thresholds were pre-registered. Section 4 presents experimental results and analysis for

each project. Section 5 discusses the meaning of the results, especially the difference between the two projects and the ability when applying in practice. Section 6 analyzes threats to validity and the limitations of the experimental design. Section 7 concludes and suggests directions for further research.

II. RELATED WORK

A. Learning-based Mutant Selection and Predictive Mutation Testing

The most common ways to reduce the cost of mutation testing is to train a learning machine model that can predict which mutant should be prioritized. Cerebro (Garg et al., 2023) use machine translation learn source code semantics to select subsuming mutant, reduce 68% equivalent mutant and 90% test excuting time compare to random selection. LEAM (Tian et al., 2022) and LEAM++ (Tian et al., 2025) train encoder-decoder on 297,029 real fault to generate high quality selective mutation fault. With Predictive Mutation Testing, Seshat (Kim et al., 2022) train DNN achive F-score with an averge 0.83 and 39 times faster than runing real test; MutatioBERT (Jain et al., 2023) fine-tune CodeBERT on aprocimately 1.9 million pair of mutant-test achive F1 0.75-0.79 same-project; SODA (Zhao et al., 2021) use contrastive learning with CodeT5 improve 5.32-114.92% kill-F1; MuDelta (Ma et al., 2021) use XGBopst on control/data flow feature achive F1 0.42 compare to 0,20 of random selection

Common limitations of this group: all of them have to be train, and they all commit that there is a lost of effeciency when transfering to a new project - MutationBERT F1 reduce from 0.75/0.79 to 0.48/0.58 and MS error increase from 0.02 to 0.05 when cross-project; SODA MS error increase from 0.0292 to 0.1242; Seshat said that the effective depend on how the project is named. This create a question: Is a training-free approach feasible?

B. Large Language Models in Mutation Testing

LLM have been applied in mutation testing on 3 main role. First is generate mutant: Wang et al. (2025) comprehensive survey and the ways to approach LLMs have an actual bug detection rate of 76.47% compare to 44.15% of rule-based technique (1,75 time of improvement), but there is a rate that mutant's generate by LLM fail to compile, duplicate and equivalence were higher by 32.73, 9.38 and 3.64 percentage points; SMART (Wang et al., 2026) use RAG and fine-tuning to pull validity from 42.89% up to 65.6% and actual bug detection rate increase to 92.61%; LLMorpheus (Tip et al., 2025) create 6.712 mutant on 13 package JavaScript which cost under 3.62 USD. Second is equivalent mutant filtering: Tian et al. (2024) achive F1 81.88% on MutantBench, take over the current technique averge 35.69% F1; GEM-LLM (Tanhaei Alizadehsani, 2025) fitering 25-30% surviving mutant which is overlooked with precision 98%. Thirty, test generating instruction: MUTGEN (Wang eet al., 2026) use Llama-3.3 70B get a averge mutation score 88.5-91.1%, pass EvoSuite; PRIMG (Bouafif et al., 2025) combine priority

mutant and generating test on Soidity, improve corrective test rate from 4.6% to 40.3%; ACH at Meta (foster et al. 2025) deloy LLM process generated 9,095 mutants and 571 tests across 10,795 Kotlin classes

Common limitations of this group: LLMs are used to generate or filter mutant, or to generate test. None of the researchs place LLMs as the role of a decision-making system that selects a subset of mutants and then directly compares it to randomness under the same budget and metric.

C. Random Selection as a Baseline and Ranking-versus-Random Evaluation

Random selection is a standard baseline because of the simplicity and reproducibility. Jimenez et al. (2018) is the closest research due to ours design: they chose mutant by naturalness by n-gram model in over 100,00 mutant for 5 Defects4J project (include Commons Lanf and Commons Math), compare with random by Wilcoxon signed-rank with effect size A_{12} . Their Result is a equavalent naturalness-based selection, a little bit worse than random, with the small consistent effect size ($A_{12} \approx 0.57 - 0.59$) - this is the source for the effect size threshold we use. Sentinel (Guizzo et al. 2022) use hyper-heuristic to achieve out-of-random evolution in 95% of cases across 40 release from 10 Java projects. Pitts (2022) analysis probabily show that at the start random select a dominator mutant with double probability but after that equivalent mutant take off the place. Zhang et al. (2019) compare 2 different metric to evaluate mutant selection on 104 Java projects and find out a very strong correlation ($R^2 = 0.8622$). About mutant bidget, AL-PMT (Borsi, 2025) report a 98% of performance, this is the best number on over 80% project when observing only 10% of mutant kill status - this is the source we choose for top 10% budget.

Common limitations of this group: the base use to compare with random is contructed good but the signal use to compare with random is still just a surface signal (n-gram) or a trained signal (hyper-heuristic, ML), it not a semantic reasoning of a training-free LLM yet.

D. Positioning

Summarizing the aboves, the gap appear: the methods of select effective mutant all need trained and decrease when cross-project (A); LLMs have prove its useful in mutation testing but only in generating and filtering mutant, not selecting (B); the ranking-versus-random assessment framework is ready but has nerr been applied to a training-free LLM (C). Due to ours survey on 34 papers - this is the first study evaluate GPT-4o-mini experiment as a mutant selector, directly compare with random fixed-seed selection under the same budget (top 10%) and the same metric (mutation score), on Defects4J with ground truth from PIT

III. METHODOLOGY

A. Dataset

We use Defects4J (Just et al., 2014), real-fault benchmark for Java, source from <https://github.com/rjust/defects4j>. From this benchmark, we choose two projects, Commons Lang and Commons Math - these projects were also used by Jumenez et al. (2018), allowing the results to be placed alongside that study.

Mutants are generated by PIT (Pitest), sourced from <https://pitest.org>, run the root test suite of each project to collect ground-truth killed/survived determination. Table 1 presents descriptive statistics of the final dataset.

TABLE I
DESCRIPTIVE STATISTICS OF THE FINAL DATASET

Project	Mutant	Version	Class	Paired group (project, version)
Commons Lang	47,727	8	149	739
Commons Math	2,967	10	18	18
Total	50,694	—	167	757

Each mutant has 10 attributes: mutant_id, project, version, class_name, method_name, line number, mutation_operator, original_code, mutated_code, surrounding_context. The mutant set spans 11 types of PIT mutation operators, of which the most common are NegateConditionalsMutator (19,524 mutants, 38.5%), MathMutator (6,233), and NullReturnValsMutator (5,810).

Class balance: The dataset is skewed towards killed mutant - 42,635 mutant killed (84.1%) compared to 8,059 survived (15.9%). This ratio remains stable between the two projects (Lang 84.2%, Math 82.8%). This is an important characteristic because the mutation score of both branches is upper-bounded by this ratio.

Scope notice: PIT is run in two different modes between two projects. Commons Lang has 5 versions run in full mode (mutating all classes, about 147 class per version, for 736 group) and 3 versions at modified mode. Commons Math runs on all 10 versions at modified/modified_fixed - PIT only mutates the classes that the Defects4J patch changes, for 1–7 classes per version and a total of only 18 groups. This scope difference is a known limitation and is analyzed in Section VI.

B. Experimental Setup and Pipeline

Model: We use GPT-4o-mini through OpenAI Chat Completions API; the actual model version returned by the API is GPT-4o-mini-2024-07-18. Configuration: temperature = 0, top_p = 1, max_tokens are not set (API defaults are used), JSON mode is not used. System prompt: "You are a careful mutation testing research assistant. Return valid JSON only."

Prompt. Ranking prompt is kept fixed for the entire experiment (quoted verbatim — in LaTeX it is included in

```
\begin{lstlisting}
You are given a list of Java mutants. Each
mutant includes a mutant ID,
```

```
mutation operator, original code, mutated code,
and surrounding code context.
```

```
Your task is to rank the mutants by usefulness
for mutation testing.
```

```
A useful mutant is a mutant that is likely to
reveal weaknesses in the test suite
and is less likely to be trivial or equivalent.
```

```
For each mutant, return:
1. mutant_id
2. usefulness_score from 1 to 5
3. short_reason
```

```
Return the final answer as JSON only, using
this format:
```

```
[
{
  "mutant_id": "M001",
  "usefulness_score": 5,
  "short_reason": "brief reason"
}
]
```

```
Do not include Markdown fences or extra
explanation outside the JSON.
```

Pipeline. The process consists of five steps (create Figure 1):

- 1) **Validate mutant pool** — checked the schema and completeness of full_sample.csv (result: 50,694 rows, status: VALID).
- 2) **GPT ranking** — grouped mutants into batches and called APIs; each mutant received a usefulness_score from 1 to 5 along with a short_reason. All 50,694 mutants were scored across 10,053 API calls. Score distribution: score 1: 343 mutants, score 2: 4,489, score 3: 14,116, score 4: 18,732, score 5: 13,014.
- 3) **Random** - random selection with random_seed = 42. Seeds for each group were derived using hashlib.sha256 instead of Python's built-in hash(), because hash() is randomly salted by process, breaking reproducibility.
- 4) **Selection under budget** — both branches select the top 10% of mutants in each group (project, version, class_name). Selection by group instead of the entire pool is mandatory: usefulness_score only has 5 levels, leading to many ties, and tie-breaks based on mutant_id in alphabetical order mean Lang_* always wins against Math_*, resulting in Math having 0 mutants selected. Each branch selects 5,438 mutants (10.7% of the pool).
- 5) **Metric computation** — calculate mutation score for each branch within each group, pair up and test (§3.3, §3.4).

Handling edge cases: Each response is checked have to have enough mutant_id that required; if model skip any ID, batch is marked and unallowed to checkpoint. All experiment have 3/10,053 fault batch (0.03%), all three cases are due to the model returning a missing mutant_id. Pipeline record checkpoint

after each success batchh adn when rerun only process mutant that have not have checkpoint, so that three fault batch are rechecked in the next run. Final result: 50.694/50.694 mutant (100%) valid point, none of the mutants is eliminated. The real API cost measured form tokens was 3,57 USD for the entire experiment (14,314,218 prompt token, 2,368,824 completion token).

Deviation compare with proposal: Batch sizes varied across all runs: 9,959 batches used 5 mutants/call (representing 98.1% of mutants), 90 batches used 10, and 3 batches used 25. Because mutants within the same batch appear in the same prompt, batch size can affect the assigned points; this effect is discussed in Section VI.

Replication package: Our replication package, including datasets, prompts, scripts, and raw results, is publicly available at: <https://github.com/KhoiPhanDinh/RT-SWT-003-nhom3>.

C. Metrics

Mutation Score (MS) is the main metric, the proportion of mutants detected by the test suite find the selected mutant set:

$MS = (\text{number of killed mutants in the selected set}) / (\text{number of selected mutants})$

MS is computed for each group (project, version, class_name), it is not a total value for the entire project. The pairing strategy follows Jimenez et al. (2018). For each group, we form a pair (MS_GPT4o, MS_random). Both approaches select the same number of mutants within each group, ensuring equal denominators and enabling a fair comparison under a fixed budget.

Ground-truth killed/survived labels are take directly from PIT reports and are deterministic; therefore, we do not have the step that measure inter-annotator agreement.

The mutant budget is set to the top 10% of the mutant pool. The threshold is motivated by AL-PMT (Borsi, 2025), which reports 98% performance achieved within 80% of cases when only 10% of mutant kill status is observed. Note that AL-PMT is an active-learning-based predictive mutation testing approach (F1/ROC-AUC), not an LLM-based selection method; therefore, the 10% value is used as a practical budget reference rather than a validated threshold for LLM-based selection.

D. Statistical Analysis Plan

Statistical test: We use Wilcoxon signed-rank test on one-sided (alternative = "greater", zero_method = "wilcox", scipy.stats.wilcoxon). Chosing reason: data are depended pairs (in the same group is measured under two selection branches), and the mutation score by group does not follow a normal distribution — the distribution is strongly skewed towards 1.0 with many duplicate values. Wilcoxon signed-rank is a non-parametric fit test for non-standard paired data, and it is the same test Jimenez et al. (2018) used for the same ranking-versus-random problem. The significance level $\alpha = 0.05$. It is a one-sided test because the hypothesis is clearly oriented: H1 states MS_GPT4o greater than MS_random, not "different".

Effect size: We report the matched-pairs rank-biserial correlation (r_{rb}), computed as:

$$r_{rb} = \frac{W^+ - W^-}{n(n+1)/2}$$

where W^+ and W^- are the sums of positive and negative ranks, respectively (ties are excluded, consistent with zero_method = "wilcox").

Threshold interpretation: $r_{rb} \geq 0.30$ for a medium effect (Cohen, 1988). The threshold is motivated by Jimenez et al. (2018), who reported that naturalness-based selection achieved a small effect size ($A_{12} \approx 0.57$ —0.58) and concluded it was "similar (slightly worse)" compared to random selection. This indicates that a small effect size is not sufficient to claim practical improvement in mutant selection.

Therefore, we require at least a medium effect to claim GPT-4o-mini has practical significance.

Success criteria: H1 is supported when all three conditions are met simultaneously: $p < 0,05$ and $r_{rb} \geq 0,30$ and median ($MS_{GPT4o} - MS_{random}$) > 0 . If there is any conditions are not satisfied, the conclusion is that there is insufficient evidence that GPT-4o-mini exceeds the random under-budget evaluation.

Sub-group analysis: Is defined before run the experiment to avoid HARKing: We ran separate validation tests for each project when $n_{group} \geq 10$; if $n_{group} < 10$, we only reported descriptive comparisons. With the data obtained, both Commons Lang ($n = 739$) and Commons Math ($n = 18$) met the criteria, so both were formally validated.

HARKing prevention: Metric, statistical test, threshold and sub-group rule were all finalized in the proposal before the experiment began and remained unchanged after the results were obtained.

REFERENCES